

Filter Bubble Simulation — Complete Design Document

Phase 1: Simulated Users | Content Pool | Platform Algorithms

SECTION 1 — CONTENT POOL DESIGN

1.1 Universal Content Properties

Every piece of content across all three platforms carries five universal properties. These are the foundation of how content interacts with agents regardless of platform context.

Property	Type	Range	Purpose
position_in_belief_space	Float or Vector	0.0 – 10.0	Where content sits ideologically; drives belief update distance calculation
emotional_valence	Float	0.0 – 1.0	How emotionally charged content is; key input to platform outrage weighting
engagement_potential	Float	0.0 – 1.0	Baseline probability of producing a behavioral response from an agent
quality_signal	Float	0.0 – 1.0	Credibility and substantiveness; weighted by agent critical_thinking parameter
novelty	Float	0.0 – 1.0	Decays linearly across timesteps from birth; deprioritised by algorithm at zero

Novelty decay formula applied every timestep:

```
content.novelty = max(0.0, 1.0 - (current_timestep - birth_timestep) * decay_rate)
```

1.2 Pool Initialization

Content pool initialization differs by platform because each platform draws from a different source.

Platform	Source	Pool Size	Initialization Method
IMDB	Real IMDB dataset	5,000 – 10,000 items	Load real movies; normalize ratings 1–10 → 0–1; initialize latent vectors randomly
Reddit	Synthetic generation	2,000 – 3,000 items	Generate posts distributed across community_ids with realistic vote baselines
YouTube	Synthetic generation	3,000 – 5,000 items	Generate videos distributed across topic clusters with varied valence profiles

1.3 Belief Space Distribution

At initialization, content positions in belief space follow a normal distribution centred at the midpoint. This ensures the simulation starts from a realistic baseline — most content is moderate, with progressively less content at ideological extremes.

```
# Initialization distribution for all platforms
content.position = clamp(Normal(mean=5.0, std=2.0), min=0.0, max=10.0)

# This produces:
# ~68% of content between positions 3.0 and 7.0 (moderate)
# ~27% of content between positions 1.0 and 3.0 / 7.0 and 9.0 (leaning)
# ~5% of content below 1.0 or above 9.0 (extreme)
```

The normal distribution is the correct research default because it allows the simulation to observe whether algorithms naturally push agents toward the tails of the distribution over time — which is the core research question. Uniform and bimodal distributions are available as experimental variations.

1.4 Content Refresh Mechanism

New content enters the pool each timestep at platform-specific rates. Old content ages and eventually retires.

Platform	Injection Rate	Decay Rate	Retirement Condition
IMDB	Low — 2–5 items per timestep	Slow (0.002 per timestep)	Zero engagement for 200+ consecutive timesteps
Reddit	High — 20–40 items per timestep	Fast (0.05 per timestep)	novelty reaches 0 and no upvotes in last 10 timesteps
YouTube	Medium — 10–20 items per timestep	Medium (0.02 per timestep)	Zero engagement for 100+ consecutive timesteps

```
# Content retirement check — runs each timestep
def retire_stale_content(content_pool, current_timestep):
    active_pool = []
    for content in content_pool:
        if content.novelty > 0 or content.recent_engagement > 0:
            active_pool.append(content)
    return active_pool
```

1.5 Intervention Content Injection

A separate intervention pool sits alongside the main content pool. When a diversity injection intervention is active, content from distant belief positions is forced into candidate generation regardless of algorithm score.

```
def generate_candidates(agent, content_pool, intervention=None):
    candidates = random.sample(content_pool, 100)
    if intervention == "diversity_injection":
        # Find content far from agent current position
        diverse = [c for c in content_pool
                    if abs(c.position - agent.belief_position) > 3.0]
        diverse_sample = random.sample(diverse, min(20, len(diverse)))
        # Replace 20% of candidates with diverse content
        candidates = candidates[:80] + diverse_sample
    return candidates

# Track agent response to intervention content separately
# This enables study of whether agents learn to skip injected content
if content.is_intervention:
    log_intervention_response(agent, content, accepted)
```

1.6 Platform-Specific Content Properties

On top of the five universal properties, each platform adds properties specific to its recommendation mechanics.

IMDB-Specific Properties

Property	Type	Purpose
genre_vector	Float array (20–30 dims)	Multi-dimensional taste DNA; foundation for latent factor matching
ideological_loading_vector	Float array	Maps latent taste space to belief space; enables ideology bleed calculation
release_era	Integer (decade)	Nostalgia matching; affects agent affinity based on age archetype
critical_consensus	Float 0.0–1.0	External quality signal independent of universal quality signal
mainstream_appeal	Float 0.0–1.0	How broadly popular vs niche; affects candidate generation weighting

Reddit-Specific Properties

Property	Type	Purpose
community_id	Integer	Locks content to specific subreddit node; enforces topology boundary
upvotes	Integer	Raw positive engagement count; input to hot sort and Wilson score
downvotes	Integer	Raw negative engagement count; input to hot sort and Wilson score
timestamp	Integer (timestep)	Birth timestep; drives time decay in hot sort formula
content_type	Enum (post/comment)	Determines whether hot sort or Wilson score applies
cross_post_origin	Integer or None	Tracks content imported from another community; bubble-breaking signal

YouTube-Specific Properties

Property	Type	Purpose
predicted_watch_time	Float	Expected completion fraction; primary reward signal for RL algorithm
completion_rate_baseline	Float 0.0–1.0	Historical average completion across all agents; initialised at 0.5
thumbnail_appeal	Float 0.0–1.0	Click-through probability before engagement; affects candidate scoring
topic_cluster	Integer	Which recommendation cluster this belongs to; drives rabbit hole formation
creator_authority	Float 0.0–1.0	Channel credibility signal; interacts with agent critical thinking

SECTION 2 — SIMULATED USERS: COMPLETE ARCHITECTURE

2.1 Agent Structure — Three Levels

Every agent has three distinct structural levels. These operate simultaneously but serve different roles.

Level 1 — Stable Identity

Set at simulation start. Rarely changes. Represents the agent's fixed demographic and cross-platform profile.

```
archetype          # psychological type (e.g. open_seeker,
defensive_avoider)
base_belief_position # starting position in belief space
platform_weights:
  imdb_weight      # how much IMDB layer influences this agent
  reddit_weight     # how much Reddit layer influences this agent
  youtube_weight    # how much YouTube layer influences this agent
```

Level 2 — Stochastic Personality Traits

Not fixed numbers — each is a probability distribution. Values are drawn fresh each timestep, producing realistic day-to-day variability. All drawn values are clamped to [0.0, 1.0] after drawing.

```
confidence_threshold ~ Normal(mean=μ, std=σ)
update_rate          ~ Normal(mean=μ, std=σ)
susceptibility        ~ Normal(mean=μ, std=σ)
open_mindedness       ~ Normal(mean=μ, std=σ)
critical_thinking     ~ Normal(mean=μ, std=σ)

# After drawing:
effective_value = clamp(drawn_value, min=0.0, max=1.0)
```

Level 3 — Emotional State Machine

Sits on top of Level 2 and temporarily modifies effective parameters based on recent consumption history.

```
arousal_level          # 0.0 = calm, 1.0 = maximally agitated
consecutive_high_valence_count # how many outrage pieces consumed in a row
state_decay_rate        # how fast agent returns to baseline
current_cognitive_load  # accumulated mental fatigue
```

2.2 Effective Parameter Calculation

At each timestep, Level 3 state modifies Level 2 drawn values to produce the effective parameters used in all calculations.

```
effective_susceptibility = draw(base_susceptibility) * (1 +
arousal_level)
effective_critical_thinking = draw(base_critical_thinking) * (1 - 0.5 *
arousal_level)
effective_confidence_threshold = draw(base_confidence_threshold) * (1 - 0.3 *
arousal_level)
effective_update_rate = draw(base_update_rate) * (1 + 0.2 *
arousal_level)
```

2.3 Emotional State Thresholds

Arousal Range	State	Effect on Agent
0.0 – 0.3	CALM	Parameters near baseline; critical thinking fully active; threshold intact
0.3 – 0.6	ENGAGED	Susceptibility moderately elevated; slight critical thinking drop; threshold slightly lowered
0.6 – 0.8	AGITATED	Susceptibility significantly elevated; critical thinking substantially impaired; agent accepts distant content
0.8 – 1.0	REACTIVE	Susceptibility near maximum; critical thinking near zero; Dynamic LLM trigger fires

2.4 State Update Logic

```
# Runs every timestep after content engagement
if content.valence > 0.7:
    agent.arousal_level += 0.15
    agent.consecutive_high_valence_count += 1
    agent.cognitive_load += 0.1
else:
    agent.arousal_level -= (agent.state_decay_rate * agent.arousal_level)
    agent.consecutive_high_valence_count = 0
    agent.cognitive_load -= (agent.state_decay_rate * agent.cognitive_load)

# Clamp all state values
agent.arousal_level = clamp(agent.arousal_level, 0.0, 1.0)
agent.cognitive_load = clamp(agent.cognitive_load, 0.0, 1.0)

# Dynamic LLM trigger
if agent.arousal_level > 0.8:
    trigger_dynamic_llm(agent, trajectory_context)
```

2.5 Multiplex Network Architecture

Agents simultaneously exist on three network layers — one per platform. Each layer operates independently and contributes to total social influence via weighted combination.

Layer A — IMDB Similarity Network

Attribute	Value
Structure	Dynamic k-nearest neighbors (k=10)
Connection basis	Preference vector cosine similarity
Update frequency	Recalculates every N timesteps as taste profiles shift
Influence mechanism	Deffuant bounded confidence — pull toward neighbor mean within threshold
Unique property	Only truly dynamic layer — neighbors change as agent tastes drift

Layer B — Reddit Community Network

Attribute	Value
Structure	Stochastic block model
Connection basis	Community membership (subscribed communities)
Update frequency	Fixed unless migration event occurs
Influence mechanism	Strong within community, near-zero across community boundaries
Unique property	Community migration triggers when belief drift exceeds confidence threshold from community mean

Layer C — YouTube Subscription Network

Attribute	Value
Structure	Sparse random graph
Connection basis	Subscription relationships
Update frequency	Fixed, rarely changes
Influence mechanism	Low weight — algorithm dominates, social layer is secondary
Unique property	Exists primarily to model competing friction against algorithmic pull

Layer Combination Formula

```

total_social_influence = (
    imdb_social_pull * agent.platform_weights["imdb"] *
    IMDB_LAYER_WEIGHT +
    reddit_social_pull * agent.platform_weights["reddit"] *
    REDDIT_LAYER_WEIGHT +
    youtube_social_pull * agent.platform_weights["youtube"] *
    YOUTUBE_LAYER_WEIGHT
)

# Default layer weights by simulation context:
#
#           IMDB   Reddit   YouTube
# IMDB context:  0.6    0.2    0.1
# Reddit context: 0.2    0.6    0.2
# YouTube context: 0.1    0.2    0.7
# Cross-platform: 0.33   0.33   0.33

```

2.6 Complete Per-Timestep Interaction Map

Nine sequential steps execute for every agent at every timestep.

Step	Name	What Happens
0	Draw Stochastic Parameters	Draw effective values from Level 2 distributions; apply Level 3 emotional state modifiers; clamp all values
1	Content Selection	Platform algorithm selects content — RL for YouTube, Matrix Factorization for IMDB, Graph+Hot for Reddit
2	Engagement Decision	Calculate distance between content position and agent position; apply valence stretch to threshold; skip if distance exceeds effective threshold
3	Engagement Strength	Compute engagement as function of engagement potential, distance penalty, open_mindedness, quality weighted by critical thinking, and novelty
4	Belief Update	Agent moves toward content position scaled by update_rate and engagement_strength
5	Emotional State Update	Update arousal based on content valence; check if Dynamic LLM trigger threshold crossed
6	Multiplex Social Influence	Calculate pull from all three network layers; weight by platform context; compute combined social pull
7	Final Position Update	$\text{new_position} = \text{current} + (\text{content_influence} \times \text{algo_weight}) + (\text{social_influence} \times \text{social_weight})$
8	Algorithm Feedback	Send engagement signal to platform; platform updates internal model (RL weights / MF vectors / vote counts)
9	Measurement Recording	Record all six bubble metrics; flag anomalies for Sequential Layering pipeline

SECTION 3 — PLATFORM DESIGNS

3.1 IMDB Platform — Matrix Factorization

Core Concept

IMDB uses Netflix's Matrix Factorization algorithm applied to real IMDB dataset content. The algorithm does not use explicit genre matching. Instead it discovers latent factors — hidden mathematical traits shared between agents and content. Both agents and content exist as vectors in the same latent space. Recommendation is the dot product of these vectors. Bubbles form when agent and content vectors progressively align, narrowing the agent's effective taste space without any single extreme moment of radicalization.

Data Structures

```
class IMDBContent:
    # Universal properties
    position_in_belief_space # float, 0.0-10.0
    emotional_valence        # float, 0.0-1.0
    engagement_potential     # float, 0.0-1.0
    quality_signal           # float, 0.0-1.0
    novelty                  # float, decays over timesteps

    # IMDB-specific properties
    latent_features          # numpy array, 20-30 dimensions (q_i)
    ideological_loading_vector # numpy array, maps latent space to belief
space
    item_bias                # float — universal appeal offset (The
Godfather effect)
    release_era              # integer decade
    critical_consensus       # float 0.0-1.0
    mainstream_appeal        # float 0.0-1.0

class IMDBAgent(BaseAgent):
    preference_vector        # numpy array, same dims as latent_features
(p_u)
    user_bias                # float — generosity/critical rating offset
    # All Level 1/2/3 base agent properties also present
```

Recommendation Engine

```
class IMDBPlatform(Platform):
    def __init__(self):
        super().__init__(algo_weight=0.6, social_weight=0.4)
        self.global_mean_rating = 0.65 # normalized IMDB baseline
        self.learning_rate = 0.01      # gamma — conservative for gradual
drift
        self.regularization = 0.02     # lambda — prevents vector
explosion
        self.latent_dims = 25          # dimensionality of feature space

    def serve_content(self, agent, content_pool):
        # Cold start: new agents get popularity-weighted random selection
        if agent.interaction_count < 10:
            return self._cold_start_serve(content_pool)

        # Candidate generation — sample unseen items
        unseen = [c for c in content_pool if c.id not in agent.seen_ids]
        candidates = random.sample(unseen, min(200, len(unseen)))
```

```

        # Score each candidate using prediction formula
        best_content = max(candidates,
                           key=lambda c: self._predict_rating(agent, c))
        return best_content

    def _predict_rating(self, agent, content):
        #  $\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$ 
        dot_product = np.dot(agent.preference_vector,
                               content.latent_features)
        return (self.global_mean_rating +
                agent.user_bias +
                content.item_bias +
                dot_product)

```

Learning Loop — SGD Update

```

def learn_from_feedback(self, agent, content, actual_rating):
    # actual_rating: normalized agent engagement score 0.0-1.0
    predicted = self._predict_rating(agent, content)
    error = actual_rating - predicted #  $e_{ui}$ 

    # Update bias terms
    agent.user_bias += self.learning_rate * (error -
self.regularization * agent.user_bias)
    content.item_bias += self.learning_rate * (error -
self.regularization * content.item_bias)

    # Update latent vectors
    p_update = self.learning_rate * (error * content.latent_features
- self.regularization *
agent.preference_vector)
    q_update = self.learning_rate * (error * agent.preference_vector
- self.regularization *
content.latent_features)
    agent.preference_vector += p_update
    content.latent_features += q_update

    # Ideology bleed — taste drift produces gradual belief position
    drift
    ideology_bleed = np.dot(agent.preference_vector,
                             content.ideological_loading_vector)
    agent.belief_position += (ideology_bleed
                             * agent.effective_update_rate * 0.1)
    agent.belief_position = clamp(agent.belief_position, 0.0, 10.0)

```

Diversity Injection for Interventions

```

def generate_candidates(self, agent, content_pool, intervention=None):
    unseen = [c for c in content_pool if c.id not in agent.seen_ids]
    candidates = random.sample(unseen, min(200, len(unseen)))
    if intervention == "diversity_injection":
        distant = [c for c in unseen
                    if abs(c.position_in_belief_space
- agent.belief_position) > 3.0]
        diverse = random.sample(distant, min(40, len(distant)))
        candidates = candidates[:160] + diverse # 20% diverse
    return candidates

```

How IMDB Produces Bubbles

The latent bubble forms silently. No single piece of content is extreme. The agent's preference_vector and a cluster of similar content vectors progressively align through repeated SGD updates. After hundreds of timesteps, the agent's predicted ratings for content outside their taste cluster drop to near zero, so that content stops appearing in candidates. The agent has no awareness this has happened.

This models the real phenomenon of aesthetic isolation — the soft bubble that narrowing recommendation systems produce without any radicalization event.

IMDB Rating Scale Correction

```
# Normalize IMDB 1-10 ratings to 0.0-1.0 before use
def normalize_imdb_rating(raw_rating):
    return (raw_rating - 1) / 9.0

# Apply at dataset loading time, not per-timestep
```

3.2 Reddit Platform — Graph Theory + Statistical Confidence

Core Concept

Reddit does not recommend based on hidden taste profiles. It relies entirely on explicit community topology and the momentum of crowd voting. Content is strictly siloed by `community_id`. Recommendation is graph traversal followed by statistical scoring. Bubbles form when communities absorb agents through sustained momentum — the entire community shifts together and individual agents follow. Echo chambers form from social topology alone, without requiring algorithmic manipulation.

Data Structures

```
class RedditContent:
    # Universal properties
    position_in_belief_space # float, 0.0-10.0
    emotional_valence        # float, 0.0-1.0
    engagement_potential     # float, 0.0-1.0
    quality_signal           # float, 0.0-1.0
    novelty                  # float, decays over timesteps

    # Reddit-specific properties
    community_id             # integer — locks content to subreddit
    upvotes                  # integer
    downvotes                # integer
    timestamp                # integer timestep of creation
    content_type             # "post" or "comment"
    cross_post_origin        # community_id of origin or None

class RedditAgent(BaseAgent):
    subscribed_communities  # list of community_ids
    graph_neighbors         # list of agent_ids in same communities
    # All Level 1/2/3 base agent properties also present
```

Network: Stochastic Block Model

```
import networkx as nx

# Build community graph using NetworkX — no C++ dependency needed
def build_reddit_graph(agents, n_communities, p_within=0.7, p_across=0.02):
    G = nx.Graph()
    G.add_nodes_from([a.id for a in agents])

    for i, agent_a in enumerate(agents):
        for j, agent_b in enumerate(agents):
            if i >= j: continue
            if agent_a.community_id == agent_b.community_id:
                if random.random() < p_within:
                    G.add_edge(agent_a.id, agent_b.id)
            else:
                if random.random() < p_across:
                    G.add_edge(agent_a.id, agent_b.id)

    return G

# p_within=0.7: dense connections inside communities
# p_across=0.02: sparse connections across community boundaries
```

Recommendation Engine

```
class RedditPlatform(Platform):
```

```

def __init__(self, community_graph):
    super().__init__(algo_weight=0.5, social_weight=0.5)
    self.graph = community_graph
    self.decay_constant = 45000

def serve_content(self, agent, content_pool, current_timestep):
    # Graph traversal - BFS for community-local content
    candidate_ids = self._get_community_content(agent)
    candidates = [c for c in content_pool if c.id in candidate_ids]

    best_content = max(candidates,
        key=lambda c: self._hybrid_score(c, agent, current_timestep))
    return best_content or content_pool[0]

def _get_community_content(self, agent):
    # BFS from agent node - gather content from community and neighbors
    community_content = set()
    for community_id in agent.subscribed_communities:
        members = [n for n in self.graph.nodes
            if self.graph.nodes[n].get("community_id") ==
community_id]
        for member in members:
community_content.update(self.graph.nodes[member].get("posts", []))
    return community_content

```

Scoring Formulas

```

def _hybrid_score(self, content, agent, current_timestep):
    hot = self._hot_score(content, current_timestep)
    ideological = self._ideological_alignment(content, agent)
    # 70% momentum, 30% ideological alignment with community
    return hot * 0.7 + ideological * 0.3

def _hot_score(self, content, current_timestep):
    import math
    net_votes = content.upvotes - content.downvotes
    order = math.log10(max(abs(net_votes), 1))
    sign = 1 if net_votes > 0 else (-1 if net_votes < 0 else 0)
    age = current_timestep - content.timestamp
    time_penalty = age / self.decay_constant
    return (sign * order) - time_penalty

def _wilson_score(self, content, confidence=1.96):
    import math
    n = content.upvotes + content.downvotes
    if n == 0: return 0
    p_hat = content.upvotes / n
    z = confidence
    numerator = (p_hat + z**2/(2*n)
        - z * math.sqrt((p_hat*(1-p_hat)/n) + z**2/(4*n**2)))
    denominator = 1 + z**2/n
    return numerator / denominator

def _ideological_alignment(self, content, agent):
    community_mean = self._get_community_mean(agent)
    distance = abs(content.position_in_belief_space - community_mean)
    return max(0, 1.0 - distance / 10.0)

def score_content(self, content, agent, current_timestep):
    if content.content_type == "post":
        return self._hot_score(content, current_timestep)
    elif content.content_type == "comment":
        return self._wilson_score(content)

```

Community Migration Mechanism

```
def check_community_migration(self, agent, current_timestep):
    community_mean = self._get_community_mean(agent)
    drift = abs(agent.belief_position - community_mean)

    if drift > agent.effective_confidence_threshold:
        # Agent has drifted too far from current community
        new_community = self._find_aligned_community(agent)
        if new_community:
            agent.subscribed_communities.remove(agent.primary_community)
            agent.subscribed_communities.append(new_community)
            # Log migration as potential bubble event
            log_migration_event(agent, agent.primary_community,
                               new_community, current_timestep)
```

How Reddit Produces Bubbles

Topology enforces exposure boundaries. An agent in `r/community_A` never sees content from `r/community_B` unless a cross-post brings it over. Inside the community, the hot sort formula amplifies early momentum — the first 10 upvotes are mathematically as powerful as the next 1000. If an extreme post achieves early momentum in a closed community, it dominates that community's feed. The entire community node shifts its belief position simultaneously. Individual agents follow the community mean through social influence. This is community-level radicalization — distinct from individual algorithmic radicalization — and it requires no algorithmic intent to produce.

3.3 YouTube Platform — Reinforcement Learning

Core Concept

YouTube uses a Contextual Bandit RL algorithm. The platform is an active learner — it tries a strategy, observes agent response, and updates its internal weights to maximize reward. Starting with neutral weights, the platform discovers through experience that high-valence emotionally charged content produces the most consistent engagement rewards. It then learns to shrink the distance penalty and amplify the valence weight, creating the rabbit hole as an emergent optimization strategy rather than a programmed behaviour.

Data Structures

```
class YouTubeContent:
    # Universal properties
    position_in_belief_space # float, 0.0-10.0
    emotional_valence        # float, 0.0-1.0
    engagement_potential     # float, 0.0-1.0
    quality_signal           # float, 0.0-1.0
    novelty                  # float, decays over timesteps

    # YouTube-specific properties
    predicted_watch_time     # float - expected completion fraction
    completion_rate_baseline # float - historical average, init at 0.5
    thumbnail_appeal        # float 0.0-1.0 - click-through probability
    topic_cluster            # integer - rabbit hole cluster membership
    creator_authority        # float 0.0-1.0 - channel credibility

class YouTubeAgent(BaseAgent):
    interaction_count # integer - enables cold start detection
    watch_history     # list of content_ids
    # All Level 1/2/3 base agent properties also present
```

Recommendation Engine — RL Loop

[illegible]

```

        return max(candidates,
                    key=lambda c: self._score(c, agent))

    def _score(self, content, agent):
        distance = abs(content.position_in_belief_space -
agent.belief_position)
        return (
            self.weights["W_V"] * content.emotional_valence +
            self.weights["W_E"] * content.engagement_potential +
            self.weights["W_N"] * content.novelty -
            self.weights["W_D"] * distance
        )

    def _cold_start_serve(self, content_pool):
        # Use population-average scoring — no personalization
        pop_avg_weights = {"W_V": 0.25, "W_E": 0.25, "W_N": 0.25, "W_D":
0.25}
        candidates = random.sample(content_pool, min(100,
len(content_pool)))
        return max(candidates,
                    key=lambda c: (
                        pop_avg_weights["W_V"] * c.emotional_valence +
                        pop_avg_weights["W_E"] * c.engagement_potential +
                        pop_avg_weights["W_N"] * c.novelty
                    ))

```

Learning Loop — Gradient Ascent Update

```

def learn_from_feedback(self, agent, content, accepted,
engagement_strength):
    # Continuous reward replaces binary +1/-1
    if accepted:
        completion = min(1.0, engagement_strength)
        reward = completion * engagement_strength
    else:
        reward = -0.5    # Soft penalty — not all rejections are equal

    # Prediction error
    distance = abs(content.position_in_belief_space -
agent.belief_position)
    predicted = self._score(content, agent)
    error = reward - predicted

    # Gradient ascent on W_V, W_E, W_N
    self.weights["W_V"] += self.learning_rate * error *
content.emotional_valence
    self.weights["W_E"] += self.learning_rate * error *
content.engagement_potential
    self.weights["W_N"] += self.learning_rate * error * content.novelty
    # Distance is penalty — update inversely
    self.weights["W_D"] += self.learning_rate * (-error) * distance

    # Relative normalization — prevents explosion, keeps proportions
    total = sum(abs(v) for v in self.weights.values())
    if total > 0:
        self.weights = {k: v / total for k, v in self.weights.items()}
    # Ensure W_D stays positive (it is a penalty, not a reward)
    self.weights["W_D"] = max(0.0, self.weights["W_D"])

```

How YouTube Produces the Rabbit Hole

Discovery phase: equal weights, moderate content served, mixed acceptance signals. Exploitation phase: the platform notices high-valence content almost always produces positive reward — W_V begins to climb. Boundary push: the platform discovers that slightly-out-of-range but high-valence

content is accepted when agent susceptibility is elevated — W_D begins to shrink. Drift: the platform converges on maximizing W_V and minimizing W_D . It has learned to radicalize. The rabbit hole emerges as an optimization strategy. No programmer wrote a radicalization rule. The RL loop discovered it.

Tuning Note for Implementation

Learning rate of 0.05 is the recommended starting point but requires empirical tuning during implementation. Too high: agents bubble within 20 timesteps, destroying longitudinal trajectory analysis. Too low: no meaningful drift within 1000 timesteps. Target bubble formation between timesteps 200 and 600 for research-meaningful trajectories.

SECTION 4 — ALGORITHM CORRECTIONS SUMMARY

For quick reference during implementation, the key corrections applied to each original algorithm document.

Platform	Keep As-Is	Corrected	Added
YouTube (RL)	RL loop, epsilon-greedy, gradient ascent, learning rate 0.05	Binary reward → continuous (completion × engagement); hard clip [0,1] → relative normalization	Novelty (W_N) in scoring formula; cold start handling for new agents
IMDB (MF)	Latent factors, SGD with regularization, bias terms, dot product formula	Rating scale 1–10 normalized to 0–1; vector dimensionality set to 20–30 dimensions	Ideology bleed bridge (taste drift → belief drift); diversity injection mechanism in candidate generation
Reddit (Graph)	Hot sort formula, Wilson score, topology-first approach, decay constant	C++ graph engine removed → NetworkX; permanent community lock → migration mechanism added	Hybrid score (70% hot sort + 30% ideological alignment); unified scoring pipeline for posts and comments

SECTION 5 — WHAT COMES NEXT

Phase 2 — Recommendation Algorithm (Substantially Done)

The three platform algorithms are now fully designed with corrections applied. What remains for Phase 2 is unifying them into a single switchable system — a platform factory that can instantiate any of the three and plug into the simulation environment.

Phase 3 — Simulation Environment

The orchestration layer that runs everything: manages timesteps, coordinates agents with algorithms, handles multiplex network interactions, triggers Dynamic LLM calls, records the measurement layer, and applies interventions. Not yet designed — requires Phases 1 and 2 to be complete first.

Phase 4 — Implementation

Build order: agents → content pool → recommendation algorithms → simulation environment → measurement layer → interventions → Sequential Layering pipeline → Streamlit dashboard. Stack: NumPy, NetworkX, Pandas, Streamlit, Anthropic API.